

Eustace the Pirate Monk — Specification

CS 2114 Project 1, Deliverable 2 — Group 87 (Marcos Salas, Aidan McIlvenni)

Java 8, default package, nothing beyond `java.util` and `student.jar`. Numbers marked *default* live in named constants; Marcos tunes them in week 2 and section 6's expected values are recomputed when they change. Attached: the full UML `docs/class-diagram.png` and the map `docs/channel-map.png`.

1. Class design

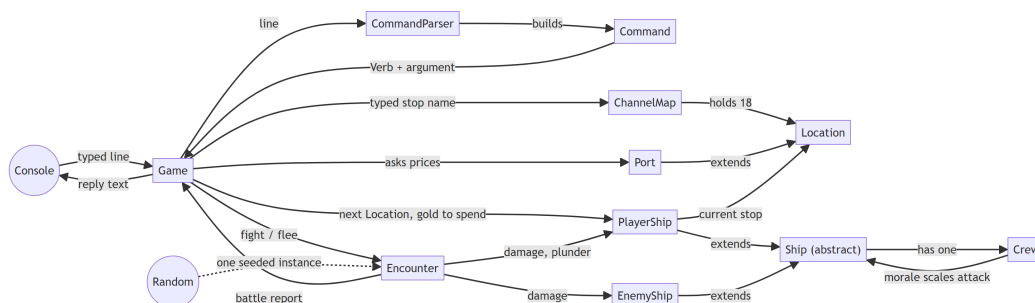
Class	Single job
Game	Runs the read–dispatch–print loop, computes the <code>GameMode</code> , and is the only class that prints. Holds <code>main</code> .
CommandParser	Turns one raw line into a <code>Command</code> ; owns every typing-slip rule (<code>DESIGN.md</code> §4 cases 1–6, 8–11, 25, 27).
Command	Immutable value: a Verb and its argument string.
ChannelMap	Builds the fixed 18-stop map and finds a stop from the name the player typed.
Location	One stop on the map: key, name, kind, and the stops one move away.
Port	A <code>Location</code> where the ship is moored: its nation and its prices.
Ship (abstract)	What both sides share: hull, cannons, armour, gold, rum, a <code>Crew</code> , and the rules for dealing and taking damage.
PlayerShip	The player's ship: where it is, notoriety, ports visited, and every purchase.
EnemyShip	A ship the player meets, built from its <code>EnemyType</code> plus the player's notoriety.
Crew	Head count, morale and greed, and the rules that move them.
Encounter	One meeting with an enemy: whether it happens, the fight round by round, fleeing, plunder.
Verb (enum)	LOOK STATUS SAIL REPAIR HIRE BUY BONUS FIGHT FLEE RETIRE HELP QUIT EMPTY UNKNOWN, each with its usage line.
GameMode (enum)	PORT SEA ENCOUNTER OVER: which verbs are legal now.
LocationKind (enum)	PORT SEA_LANE HIGH_SEAS, each with its encounter chance.
Nation (enum)	ENGLAND FRANCE PIRATE.
EnemyType (enum)	MERCHANT PIRATE COAST_GUARD, each with base hull, crew, cannons, armour, gold, morale and notoriety gain.

No class both parses text and changes ship values; no class both holds game state and prints it. Course concepts used: an abstract class with two subclasses overriding `describe()`, a second inheritance pair (`Port` extends `Location`), enums that carry data, `ArrayList` holding a graph as adjacency lists, one injected `Random` so JUnit replays a run, and one `student.TestCase` class per production class.

Why this split. Both sides fight by the same rules, so `Encounter` calls `attackStrength()` and `takeDamage()` on a `Ship` without knowing which side it is. `Crew` is separate because morale scales attack for both sides and a broken crew ends a fight (enemy surrenders) or the run (player mutinies). Cannons and armour are levels 1–5, so buy has three fixed words and no port is out of stock; all ports share one price list, Sark halving rum. The mode is never stored: `mode()` returns `OVER` after the run ends, `ENCOUNTER` when `encounter != null`, `PORT` when the ship's stop is a port, else `SEA`.

The map. Seven ports (three English, three French, Sark the pirate home), eight sea lanes each joining two ports, three high-seas stretches touching the nearest ports. One sail moves one stop: Dover to Boulogne is `sail dover boulogne` then `sail boulogne`. Keys have no spaces; `find` hyphenates the typed words and, failing that, tries the first word alone. Each move into a sea stop makes the crew drink and rolls one encounter.

2. System diagram



Boxes are the classes above (enums omitted); each arrow points the way the labelled thing travels. One turn: Game reads a line, CommandParser returns a Command, Game checks the verb against mode(), calls the object for that verb, and prints what comes back.

3. Data & state

All fields private; final where marked; the ranges hold after every public method.

Game: Scanner in; final ChannelMap map; final PlayerShip player; Encounter encounter (null unless a fight is on); final Random random, the only one in the program; boolean running; static final int GOLD_TARGET = 1000 *default*. **CommandParser:** no fields. **Command:** final Verb verb; final String argument, lower case, single-spaced, quotes and ., !? stripped from each word's ends, "" when absent.

ChannelMap: ArrayList<Location> stops (18, unique keys, fixed after standard()); Port home. An ArrayList because the only operation is "find the stop whose key was typed", at most 18 equals calls per sail. **Location:** final String key, name; final LocationKind kind; ArrayList<Location> neighbours (2–4; if A lists B then B lists A; never itself or a duplicate; getNeighbours() returns an unmodifiable view). An adjacency list: an 18×18 matrix would hold 324 entries for 25 links. **Port** adds final Nation nation and REPAIR_PRICE = 2, HIRE_PRICE = 15, RUM_PRICE = 4, UPGRADE_PRICE = 100 *default*, gold per unit; level *n* costs UPGRADE_PRICE × *n*. **LocationKind:** final int encounterPercent: PORT 0, SEA_LANE 35, HIGH_SEAS 60 *default*.

Ship: final String name; int hull (0..maxHull); final int maxHull; int cannons (1..5); int armour (0..5); int gold, int rum (never negative); final Crew crew. Subclasses change hull, cannons and armour only through protected setters. **PlayerShip** adds Location location (never null), int notoriety, int portsVisited, static final int MAX_LEVEL = 5; starts *default* at hull 100, crew 20 at morale 70, cannons 1, armour 1, gold 200, rum 30, at Sark. **EnemyShip** adds final EnemyType type; for notoriety *n*: hull = base + 2*n*, crew = base + *n*/2 (≤ 40), cannons and armour = base + *n*/10 (≤ 5), gold = base + 5*n*, rum = crew, morale = base. **EnemyType** *default*: MERCHANT 40 hull, 8 crew, 1 cannon, 0 armour, 150 gold, morale 30, gain 1; PIRATE 60, 15, 2, 1, 100, 60, 2; COAST_GUARD 80, 20, 3, 2, 50, 80, 3. **Crew:** int count (0..MAX_COUNT = 40); int morale, int greed (0..100; greed starts 0 and only the player's crew gains it). *Default* steps: −5 morale per man lost, −10 when rum runs out, +10 on a win, −15 on a flee, +1 greed per 20 gold plundered. **Encounter:** final PlayerShip player; final EnemyShip enemy; final Random random; FLEE_DAMAGE = 10, ROLL_RANGE = 6 *default*.

How the numbers move. moraleFactor() = 0.5 + morale/200. attackStrength() = (5 × cannons + crew/2) × moraleFactor, floored, at least 1 (fresh player 12). takeDamage(raw) lands raw − 2 × armour, at least 1 when raw > 0: hull drops by that and the crew loses landed/5 men. isDefeated() = hull 0, crew 0, or crew.isBroken() (morale 0 or greed 100). A fight round: the player fires attackStrength() + random.nextInt(ROLL_RANGE); if the enemy stands, it fires back the same way; repeat until one side is defeated. A win moves the enemy's gold and rum across, adds the type's notoriety gain, and calls crew.onWin(gold). Each sail into a sea stop: the crew needs (count + 9)/10 bottles and drinks them, or drinks what is left and loses 10 morale; then roll draws nextInt(100) against the stop's chance, and a second draw picks the type (sea lane: < 70 merchant, else coast guard; high seas: < 50 pirate, else merchant). A purchase buys the smallest of the amount typed, gold ÷ price and the room left, then charges units × price.

4. Method signatures

"Refuses" = returns 0 or false and changes nothing; dispatch turns each refusal into the section 5 message. *IAE* = throws IllegalArgumentException.

Game: Game(Scanner in, ChannelMap map, Random random). static void main(String[] args) builds a game on System.in, ChannelMap.standard(), new Random() and calls run. void run() prints the opening scene, then loops read → parse → dispatch → print until mode() is OVER. String dispatch(Command c) applies one command in the current mode and returns the text to print. GameMode mode(), PlayerShip getPlayer(), Encounter getEncounter().

CommandParser: Command parse(String line) normalises and returns a Command with verb EMPTY for blank input, UNKNOWN when the first word is no verb, else the verb and the rest as argument. static int parseCount(String text) returns an int ≥ 0 or throws NumberFormatException for "ten", "10.1", "-5", "", or past int range. **Command:** Command(Verb verb, String argument); Verb getVerb(); String getArgument(); String getWord(int i), the *i*-th argument word or ""; String toString() gives "sail dover".

ChannelMap: static ChannelMap standard() builds the 18 stops and 25 links; void add(Location stop), IAE on a duplicate key; void connect(String keyA, String keyB) links both ways, IAE on an unknown key; Location find(String name) returns the stop or null; Port getHome(). **Location:** Location(String key, String name, LocationKind kind); void connect(Location other) adds each to the other once, IAE on self; boolean isNextTo(Location other); List<Location> getNeighbours(); getters; String describe() gives name, kind, neighbours. **Port:** Port(String key,

String name, Nation nation); Nation getNation(); int rumPrice(), half at Sark; int upgradePrice(int nextLevel); describe() adds nation and prices.

Ship: Ship(String name, int maxHull, int cannons, int armour, Crew crew, int gold, int rum); int attackStrength(); int takeDamage(int raw) returns hull lost; boolean isDefeated(); void addGold(int), void spendGold(int) (IAE beyond what is held), void addRum(int), int takeRum(int) returns bottles taken, capped at the hold; Crew getCrew(); a getter per field; abstract String describe(). **PlayerShip:** PlayerShip(String name, Port home); void moveTo(Location d), IAE unless d is a neighbour, counts a port arrival; int repair(int points, int pricePerPoint), int hire(int men, int pricePerMan), int buyRum(int bottles, int pricePerBottle) buy up to the amount, capped by gold and room, and return units bought; boolean upgradeCannons(int price), boolean upgradeArmour(int price) refuse at MAX_LEVEL or without the gold; boolean payBonus(int gold) refuses beyond what is held, else hands the gold to the crew; void addNotoriety(int); getters; describe() is the status report. **EnemyShip:** EnemyShip(EnemyType type, int notoriety); EnemyType getType(); describe(), what the lookout sees.

Crew: Crew(int count, int morale) caps both; int lose(int men), int hire(int men) return the number removed or added after capping; double moraleFactor(); int drink(int bottlesAvailable) returns bottles drunk, dropping morale when short; void onWin(int plunder); void onFlee(); void receiveBonus(int gold) lowers greed by gold per head and raises morale by half that; boolean isBroken(); getters. **Encounter:** Encounter(PlayerShip player, EnemyShip enemy, Random random); static Encounter roll(Location where, PlayerShip player, Random random) returns a new encounter or null; String fight() resolves the whole battle and returns the narration; String flee() applies FLEE_DAMAGE and onFlee(); EnemyShip getEnemy(); String describe(). **Enums:** Verb.getUsage(), LocationKind.getEncounterPercent(), a getter per EnemyType field. parse calls Verb.valueOf and turns its exception into UNKNOWN.

5. Where validation lives

Case numbers are DESIGN.md §4. Every response leaves all fields unchanged unless the row says otherwise. dispatch checks the mode first: FIGHT/FLEE need ENCOUNTER, REPAIR/HIRE/BUY/BONUS need PORT, SAIL is refused in ENCOUNTER.

#	Case	Caught in	Response
1	Blank line	parse → EMPTY	dispatch returns ""
2	Unknown verb	parse → UNKNOWN	"I don't understand 'sial'. Type help."
3	Missing argument	dispatch: argument ""	that verb's usage line
4	Extra words	find falls back to the first word; amounts use getWord(0)	echoes sail dover, proceeds
5, 6, 25	Case, spacing, quotes, punctuation, invisible characters	parse: strip, toLowerCase, split on \s+, trim "' . , ! ?	treated as canonical
7	Real stop, not adjacent	dispatch(SAIL): find non-null, isNextTo false	lists the neighbours
8	sail help	parse reads the verb from the first word only	find("help") null → "no such place"
9, 11	Negative, ten, 10.1	parseCount throws; dispatch catches	"give a whole number, 0 or more"
10	Zero	parseCount → 0; purchases return 0	"bought 0"
12	Overflow	PlayerShip divides gold by price before multiplying	buys what gold covers; no negative total
13	More than affordable or holdable	purchases cap by gold, maxHull, MAX_COUNT	reports units bought (state changes by that much)
14, 15	Wrong mode	dispatch mode check	"nothing to fight" / "you must be in port"
16	Unknown item	dispatch(BUY): word not cannons, armour, rum	"ports sell cannons, armour and rum"
17	retire elsewhere or poor	dispatch: stop != home or gold < GOLD_TARGET	prints gold still needed
18	Quit confirmation	dispatch(QUIT) reads one more line; only y/yes ends	anything else returns to the prompt
19, 20	Spam look/status; sail <current>	those branches never call roll or drink; dispatch(SAIL) refuses find(arg) == location	no clock, no roll; "you are already there"
21	Repeated flee	flee always applies FLEE_DAMAGE	hull drains until isDefeated ends the run
22	Upgrade at max	upgradeCannons/upgradeArmour check the level before gold	refused, gold untouched

#	Case	Caught in	Response
23, 27	100 000-character line; injection	parse compares the first word with Verb names only; find and BUY compare with fixed keys and three item words	UNKNOWN; text is data, never executed
24	Input ends	run checks hasNextLine() before every read	prints "input ended", running = false
26	Accents, emoji	find null; item word matches nothing	case 8 or case 16 message

6. Test plan

One student. TestCase class per production class, in the default package so tests reach the protected setters. FixedRandom is a test-only Random whose nextInt returns a set value (0 forces an encounter, the first-listed type and a zero damage roll; 99 forces none). Fresh = a new PlayerShip at Sark with the section 3 start values. A method with no parameters gets an edge state (E); getters are covered by constructor rows. Bracketed numbers are DESIGN.md §4 cases; all 27 appear.

Method	Normal → expected	Bad input or edge → expected
Game(...), run	fresh game → mode() PORT, at Sark; script look, status, quit, yes → output holds "Sark", "Hull 100/100"; OVER	null map → IAE; blank line, sial, end of input → unknown-command text, "input ended", no exception (1, 2, 24)
dispatch(SAIL)	sail barfleur sark → at the lane, rum 28, no encounter	sail → usage (3); sail dover → names Barfleur–Sark, West Channel (7); sail sark → "already there" (20)
dispatch(HIRE)	hire 5 → crew 25, gold 125	hire 2147483647 → crew 33, gold 5 (12, 13); at sea → "must be in port" (15)
dispatch(BUY)	buy rum 20 at Sark → rum 50, gold 160	buy spyglass → item list (16); setCannons(5), buy cannons → refused, gold 200 (22)
dispatch(FIGHT), (FLEE)	FixedRandom(0), sail barfleur sark, fight → merchant surrenders; gold 350, rum 36, notoriety 1, encounter null. Same setup, flee → hull 92, crew 19, morale 50	in port → "nothing to fight" (14); flee with setHull(5) → hull 0, OVER, "sank" (21)
dispatch(RETIRE)	addGold(800), retire → OVER, text shows 1000 gold	addGold(200) → "600 more gold needed", still running (17)
dispatch(QUIT) via run	quit, yes → OVER	quit, yse → still running (18)
LOOK, STATUS, HELP	text holds the stop name / "Morale 70" / every usage	three looks at sea → no encounter (19); help sail → same text (4)
parse, parseCount	SAIL "Dover". → SAIL, dover (5, 6, 25); "25" → 25; "0" → 0 (10)	"\t \n" → EMPTY (1); sial, System.exit(0) → UNKNOWN (2, 27); -50, ten, 10.1, 9999999999 → NumberFormatException (9, 11)
Command(...), getWord, toString	(BUY, "rum 20"): getWord(1) "20"; "buy rum 20"	(null, "x") → IAE; getWord(5) → ""
ChannelMap.standard, add, connect, find	18 stops, home sark, every link both ways; dover boulogne → the lane; dover now please → Dover (4)	duplicate key → IAE; connect("dover", "atlantis") → IAE; atlantis, help, dövér → null (8, 26)
Location(...), connect, isNextTo, getNeighbours, describe	a.connect(b) → a.isNextTo(b); Sark lists Barfleur–Sark, West Channel in that order; text holds both	a.connect(a) → IAE; twice → listed once; E add on the list → UnsupportedOperationException
Port(...), rumPrice, upgradePrice, describe	Dover → ENGLAND, 4, upgradePrice(2) 200; text holds prices	E Sark → rum 2
Ship(...) (test subclass), attackStrength, isDefeated	maxHull 40 → hull 40; fresh → 12, not defeated	maxHull 0 or null crew → IAE; E morale 0 → 7; setHull(0) → defeated
takeDamage, addGold, spendGold, addRum, takeRum	fresh, takeDamage(12) → 10; hull 90, crew 18, morale 60. Gold 50 → 250 / 150; rum 10 → 40; takeRum(5) → 5	negatives → IAE; takeDamage(500) → 100, hull 0; spendGold(500) → IAE, gold 200; takeRum(50) → 30, rum 0
PlayerShip(...), moveTo, addNotoriety	at Sark, notoriety 0; Sark → lane → Sark: ports visited 1; addNotoriety(2) → 2	null port → IAE; Sark → Dover → IAE, unmoved; -1 → IAE
repair, hire, buyRum	setHull(80), repair(10, 2) → 10, gold 180; hire(5, 15) → 5; buyRum(20, 2) → 20	repair(50, 2) → 20 (13); hire(30, 15) → 13, gold 5 (12); buyRum(20, 4) with 3 gold → 0
upgradeCannons, upgradeArmour, payBonus, describe	upgrade → true, level 2, gold 0; after onWin(600), payBonus(100) → greed 25, morale 82; describe holds every field	level 5 → false (22); 50 gold → false; payBonus(500) → false, unchanged
EnemyShip(...), describe	(MERCHANT, 0) → hull 40, crew 8, gold 150, morale 30; (MERCHANT, 10) → 60, 13, cannons 2, gold 200	(PIRATE, -1), (null, 0) → IAE

Method	Normal → expected	Bad input or edge → expected
Crew(...), lose, hire, moraleFactor, isBroken	(20, 70) → factor 0.85, not broken; lose(2) → 2, count 18, morale 60; hire(5) → 5	(50, 70) → 40; lose(25) → 20, morale 0, broken; hire(30) → 20 (13); negatives → IAE
drink, onWin, onFlee, receiveBonus	30 bottles → 2, morale 70; onWin(150) → 80, greed 7; onFlee → 55; bonus 100 after onWin(600) → greed 25, morale 82	drink(1) → 1, morale 60; onWin(5000) → greed 100; negatives → IAE
Encounter(...), roll, describe	sea lane, FixedRandom(0) → merchant; text holds "merchant"	null enemy → IAE; port → null; high seas with 99 → null
fight, flee	fresh vs (MERCHANT, 0), roll 0 → surrender in round 3; hull 97, gold 350, rum 38, morale 80, greed 7. flee → hull 92, crew 19, morale 50	E setHull(5): vs (PIRATE, 0) → defeated round 1, gold 200, "sank"; flee → hull 0, defeated (21)
enum getters	SAIL → sail <stop>; SEA_LANE → 35; MERCHANT gain 1	E every Verb usage non-empty; PORT → 0

7. Division of work

Owner	Classes	Also
Aidan	CommandParser, Command, Verb, GameMode, Ship, PlayerShip, EnemyShip, EnemyType, Crew, Encounter	cases 1–6, 8–13, 18, 21–27; final Web-CAT submission
Marcos	Game, ChannelMap, Location, Port, LocationKind, Nation	the map, all <i>default</i> numbers, cases 7, 14–17, 19–20; play-through transcript for the oral defence

Day one, together: every class with its section 4 signatures and stub bodies, so the project compiles and each person edits only their own files. Week 1 milestone (Tuesday 7 pm): start at Sark, look, status, sail round all 18 stops, repair, hire, buy, help, quit. Week 2: fights, crew rules, bonus, retire, mutiny and sinking; both play to set the numbers. Game is the only class calling both people's code and Marcos owns it, so nobody edits the same file; each class lands on main with its test class and the merged build must pass Web-CAT with no style warnings. If time runs short, cut in this order, each leaving every other class unchanged: greed and bonus; rum, with a fixed morale drop per sea move; the three high-seas stops.

8. Revised scope

Change	Why	Origin
Port map became an 18-stop metro map (ports, sea lanes, high seas), each kind with its own encounter chance; sail moves one stop; ports named, Sark as home	The route chosen changes the risk; one stop per move needs no route-finding	Marcos; a 25-stop draft lost its coastal-waters stops so a crossing is two moves (group discussion); names from GenAI draft
Map stored as an ArrayList of stops with a neighbour list per stop	Closes the \$5 unknown on coding the map	Aidan's \$5 idea; collection choice from GenAI review
Crew morale, greed and rum moved from stretch goal 3 into the MVP in basic form (DESIGN.md FR15–17); bonus added	Morale drives combat for both sides and gives fights two endings	Marcos; basic/full split from GenAI review
Player and enemy share an abstract Ship; enemies are EnemyType × notoriety	One fight routine; difficulty scaling from DESIGN.md §1 returns	Marcos asked for a shared base; GenAI draft chose inheritance
Weapon items and per-port stock became cannon/armour levels and one price list	Removes an Item class and seven stock lists	GenAI review, after Marcos set a two-week budget
Overflow (case 12) handled by dividing before multiplying; refusals are return values and dispatch writes every message; parse never throws	No ArithmeticException or custom exception to catch; one place makes player-facing text	Aidan's draft; division from GenAI review
Mode computed by Game.mode(); OVER is a mode value	Closes the \$5 unknown on restricting commands by mode	Aidan's GameMode plus the "state machine" note in DESIGN.md
One Random injected through Game; Encounter fully constructible	Closes the \$5 unknown on forcing a fight in JUnit	GenAI review of \$5, both drafts